

COMP9312 Project — Q2

An Index-Based Solution for k -Triangle Core Search

YUHUI LUO (z5565446)

1. Problem

In a simple undirected graph G , a *triangle* is a set of three pairwise adjacent vertices. A **k -triangle core** is a maximal connected induced subgraph in which every vertex lies in at least k triangles *inside that subgraph*. Given k and a vertex v , $\text{query}(k, v)$ returns every vertex of the k -triangle core containing v . We build an index once and then answer many queries.

2. Design Overview

The k -triangle core is the **vertex analogue of the k -core**, replacing “degree” by “number of triangles a vertex lies in”. Two ingredients are needed: (i) how deep each vertex is — its *triangle-core number* $tc(v)$ — and (ii) the right notion of connectivity.

2.1 Why plain connectivity fails: the bridge

Consider Figure 1: a K_4 on $\{0, 1, 2, 3\}$, a triangle $\{3, 4, 5\}$, a bridge $5-6$, and a K_4 on $\{6, 7, 8, 9\}$. Here $\text{query}(1, 0) = \{0, 1, 2, 3, 4, 5\}$, *not* $\{0, \dots, 9\}$ — even though the graph is connected and every vertex lies in a triangle. The bridge edge $(5, 6)$ belongs to *no* triangle, so it must not join the two sides. The correct relation is **triangle connectivity**: vertices are in the same k -core only when linked by a chain of triangles (consecutive ones sharing a vertex) whose vertices all survive to level k . We encode this with an **edge level**

$$\text{level}(u, w) = \max_{\text{triangles } (u, w, x)} \min(tc(u), tc(w), tc(x)),$$

the highest k at which edge (u, w) sits inside a triangle whose three vertices all have $tc \geq k$; an edge in no triangle has level 0. Then

$$\text{query}(k, v) = \begin{cases} [] & \text{if } tc(v) < k, \\ \{\text{vertices reachable from } v \text{ via edges with level } \geq k\} & \text{otherwise,} \end{cases}$$

which is exactly the triangle-connected k -core component of v .

3. Computing $tc(v)$: Triangle-Core Peeling

$tc(v)$ is obtained by a **peeling** (degeneracy) process on triangle counts, exactly analogous to k -core decomposition:

1. Compute $t(v)$ = number of triangles containing v in G .
2. Repeatedly remove the vertex v of *smallest* current $t(v)$. Removing v destroys every triangle through it, so for each such triangle (v, a, b) decrement $t(a)$ and $t(b)$.
3. $tc(v)$ is the value of $t(v)$ when v is removed (the removal order keeps it monotone — the “max with running level” rule).

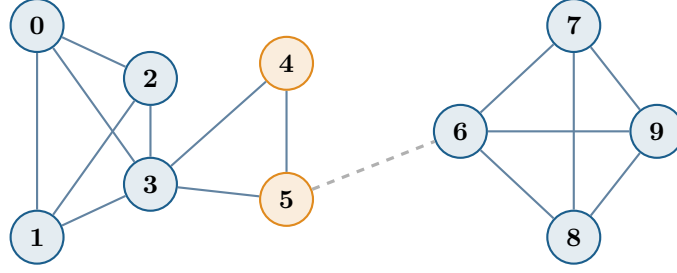


Figure 1: Spec Figure 1. ■ $tc = 3$ (each vertex is in a K_4); ■ $tc = 1$ (only in $\{3, 4, 5\}$). The dashed bridge $(5, 6)$ is in no triangle, so it has level 0 and never connects the two cores.

We keep vertices bucketed by current triangle count in the **Batagelj–Zaversnik** array structure, so each removal and each triangle destruction is $O(1)$; peeling is $O(n + T)$, where T is the number of triangles.

Triangle enumeration. Orient each edge from the lower- to the higher-rank endpoint, $\text{rank} = (d(\cdot), \text{id})$. Intersecting forward-neighbour lists then reports each triangle once, from its lowest-rank vertex, in $\sum_{(u,w) \in E} \min(d(u), d(w)) = O(m^{1.5})$ time.

4. Index and Query

`build()` stores two arrays:

- $tc[v]$ for every vertex — $O(n)$;
- `level[]`, laid out *parallel to* `graph.indices` (the flat adjacency array), so that a query is a plain threshold traversal — $O(m)$.

`query( $k, v$ )` returns `[]` if $tc(v) < k$; otherwise it runs a DFS from v that crosses only edges with $\text{level} \geq k$ (whose endpoints then automatically satisfy $tc \geq k$) and returns the visited set.

4.1 Worked Example (Figure 1)

$tc = 3$ on $\{0, 1, 2, 3, 6, 7, 8, 9\}$ and $tc = 1$ on $\{4, 5\}$.

- `query(1, 0)`: the K_4 edges and $(3, 4), (3, 5), (4, 5)$ all have level 1 (triangle $\{3, 4, 5\}$ has level $\min(3, 1, 1) = 1$); the bridge $(5, 6)$ has level 0. $\Rightarrow \{0, 1, 2, 3, 4, 5\}$.
- `query(3, 3)`: edges $(3, 4), (3, 5)$ have level $1 < 3$, so 4, 5 are excluded; only the K_4 edges (level 3) are crossed. $\Rightarrow \{0, 1, 2, 3\}$.
- `query(4, 0)`: $tc(0) = 3 < 4 \Rightarrow []$.

Result. `query(1, 0) = {0, 1, 2, 3, 4, 5}`, `query(3, 3) = {0, 1, 2, 3}`, `query(4, 0) = []`.

5. Correctness

- **Peeling yields tc .** Identical to k -core correctness with “triangle count” as the monotone quantity: always removing the current minimum guarantees no later-removed vertex has a smaller core number, so each vertex’s removal value equals $tc(v)$.

- **Edge level captures triangle connectivity.** If a triangle (a, b, c) has $\min(tc(a), tc(b), tc(c)) \geq k$, then all three of its edges get level $\geq k$, making its vertices mutually reachable at level k ; conversely an edge reaches level k only through such a triangle. Chaining triangles that share a vertex therefore chains the DFS, so the reachable set is exactly the triangle-connected k -core component.
- **v is always included.** $tc(v) \geq k$ means v lies in a triangle of level $\geq k$, hence has an incident edge of level $\geq k$ and is reached.

6. Complexity Analysis

Let n be the number of vertices, m the number of edges, $d(v)$ the degree of v , T the number of triangles ($T = O(m^{1.5})$), and C the vertex set returned by a query.

6.1 Index construction time: $O(m^{1.5} + n)$

Stage	Cost
Triangle listing (forward, degree-ordered)	$\sum_{(u,w) \in E} \min(d(u), d(w)) = O(m^{1.5})$
Peeling (Batagelj–Zaversnik on triangle counts)	$O(n + T)$
Edge levels (aggregate + scatter)	$O(T) + O(n + m)$
Total	$O(m^{1.5} + n + m + T) = O(m^{1.5} + n)$

since $T = O(m^{1.5})$ and $m = O(m^{1.5})$.

6.2 Query time: $O(\sum_{x \in C} d(x) + |C| \log |C|)$

A query is a single DFS over the induced subgraph of the answer, scanning each returned vertex's adjacency once ($\sum_{x \in C} d(x)$), followed by an $O(|C| \log |C|)$ sort that returns the vertices in canonical ascending order. Both terms are **output-sensitive** — bounded by the returned component — and the traversal term dominates in a genuine triangle core (every returned vertex has degree ≥ 2), so the query is effectively output-linear and $O(n + m)$ in the worst case. The initial $tc(v) < k$ rejection is $O(1)$.

6.3 Space: $O(n + m)$

The persistent index is $tc[]$ ($O(n)$) plus $level[]$ ($O(m)$), so $O(n + m)$. Construction additionally uses $O(n + T)$ transient memory (the triangle list and per-vertex triangle incidence), which may be released once `build()` finishes.

7. Key Implementation Excerpt

```
# --- peeling: tc(v) via Batagelj-Zaversnik buckets on triangle counts ---
for i in range(n):
    v = order[i]; lvl = d[v]; tc[v] = lvl # smallest remaining count
    for t in tri_of[v]: # triangles through v
        if not alive[t]: continue
        alive[t] = 0 # triangle dies with v
        for w in triangles[t]:
            dw = d[w]
```

```
    if w == v or dw <= lvl: continue # w already fixed, leave it
    p, s = pos[w], head[dw] # move w one bucket down
    f = order[s]
    if f != w:
        order[p], order[s] = f, w; pos[f], pos[w] = p, s
    head[dw] += 1; d[w] = dw - 1

# --- query: threshold DFS over edge levels ---
def query(self, k, v):
    if self.tc[v] < k: return []
    off, adj, level = self.graph.offsets, self.graph.indices, self.level
    seen = {v}; stack = [v]
    while stack:
        x = stack.pop()
        for i in range(off[x], off[x + 1]):
            if level[i] >= k and adj[i] not in seen:
                seen.add(adj[i]); stack.append(adj[i])
    return sorted(seen) # canonical (ascending) vertex order
```